

```
! No user data
ethicalads:
  topic: devs
  region: global
  type: image
```

Reach the right audience on a privacy-first ad network only for software devs: **EthicalAds**

www.ethicalads.io

Ads by EthicalAds

Learning to Rank

Contents

- Overview
- Training with the Pairwise Objective
- Position Bias
- Loss
- Constructing Pairs
- Obtaining Good Result
- Distributed Training
- Reproducible Result
- References

Overview

Often in the context of information retrieval, learning-to-rank aims to train a model that arranges a set of query results into an ordered list [1]. For supervised learning-to-rank, the predictors are sample documents encoded as feature matrix, and the labels are relevance degree for each sample. Relevance degree can be multi-level (graded) or binary (relevant or not). The training samples are often grouped by their query index with each query group containing multiple query results.

XGBoost implements learning to rank through a set of objective functions and performance metrics. The default objective is `rank:ndcg` based on the `LambdaMART` [2] algorithm, which in turn is an adaptation of the `LambdaRank` [3] framework to gradient boosting trees. For a history and a summary of the algorithm, see [5]. The implementation in XGBoost features deterministic GPU computation, distributed training, position debiasing and two different pair construction strategies.

Training with the Pairwise Objective

`LambdaMART` is a pairwise ranking model, meaning that it compares the relevance degree for every pair of samples in a query group and calculate a proxy gradient for each pair. The default objective `rank:ndcg` is using the surrogate gradient derived from the `ndcg` metric. To train a XGBoost model, we need an additional sorted array called `qid` for specifying the query group of input samples. An example input would look like this:

QID	Label	Features
1	0	x_1
1	1	x_2
1	0	x_3
2	0	x_4
2	1	x_5
2	1	x_6
2	1	x_7

Notice that the samples are sorted based on their query index in a non-decreasing order. In the above example, the first three samples belong to the first query and the next four samples belong to the second. For the sake of simplicity, we will use a synthetic binary learning-to-rank dataset in the following code snippets, with binary labels representing whether the result is relevant or not, and randomly assign the query group index to each sample. For an example that uses a real world dataset, please see [Getting started with learning to rank](#).

```
from sklearn.datasets import make_classification
import numpy as np

import xgboost as xgb

# Make a synthetic ranking dataset for demonstration
seed = 1994
X, y = make_classification(random_state=seed)
rng = np.random.default_rng(seed)
n_query_groups = 3
qid = rng.integers(0, n_query_groups, size=X.shape[0])

# Sort the inputs based on query index
sorted_idx = np.argsort(qid)
X = X[sorted_idx, :]
y = y[sorted_idx]
qid = qid[sorted_idx]
```

The simplest way to train a ranking model is by using the scikit-learn estimator interface. Continuing the previous snippet, we can train a simple ranking model without tuning:

```
ranker = xgb.XGBRanker(tree_method="hist", lambdarank_num_pair_per_sample=8, objective="rank:ndcg",
lambdarank_pair_method="topk")
ranker.fit(X, y, qid=qid)
```

Please note that, as of writing, there's no learning-to-rank interface in scikit-learn. As a result, the `xgboost.XGBRanker` class does not fully conform the scikit-learn estimator guideline and can not be directly used with some of its utility functions. For instances, the `auc_score` and `ndcg_score` in scikit-learn don't consider query group information nor the pairwise loss. Most of the metrics are implemented as part of XGBoost, but to use scikit-learn utilities like `sklearn.model_selection.cross_validation()`, we need to make some adjustments in order to pass the `qid` as an additional parameter for `xgboost.XGBRanker.score()`. Given a data frame `X` (either pandas or cuDF), add the column `qid` as follows:

```
import pandas as pd

# 'X', 'qid', and 'y' are from the previous snippet, they are all sorted by the 'sorted_idx'.
df = pd.DataFrame(X, columns=[str(i) for i in range(X.shape[1])])
df["qid"] = qid

ranker.fit(df, y) # No need to pass qid as a separate argument

from sklearn.model_selection import StratifiedGroupKFold, cross_val_score
# Works with cv in scikit-Learn, along with HPO utilities like GridSearchCV
kfold = StratifiedGroupKFold(shuffle=False)
cross_val_score(ranker, df, y, cv=kfold, groups=df.qid)
```

The above snippets build a model using `LambdaMART` with the `NDCG@8` metric. The outputs of a ranker are relevance scores:

```
scores = ranker.predict(X)
sorted_idx = np.argsort(scores)[::-1]
# Sort the relevance scores from most relevant to least relevant
scores = scores[sorted_idx]
```

Position Bias

Added in version 2.0.0.

 Note

The feature is considered experimental. This is a heated research area, and your input is much appreciated!

Obtaining real relevance degrees for query results is an expensive and strenuous, requiring human labelers to label all results one by one. When such labeling task is infeasible, we might want to train the learning-to-rank model on user click data instead, as it is relatively easy to collect. Another advantage of using click data directly is that it can reflect the most up-to-date user preferences [1]. However, user clicks are often biased, as users tend to choose results that are displayed in higher positions. User clicks are also noisy, where users might accidentally click on irrelevant documents. To ameliorate these issues, XGBoost implements the `Unbiased LambdaMART` [4] algorithm to debias the position-dependent click data. The feature can be enabled by the `lambdarank_unbiased` parameter; see [Parameters for learning to rank](#) (`rank:ndcg`, `rank:map`, `rank:pairwise`) for related options and [Getting started with learning to rank](#) for a worked example with simulated user clicks.

Loss

XGBoost implements different `LambdaMART` objectives based on different metrics. We list them here as a reference. Other than those used as objective function, XGBoost also implements metrics like `pre` (for precision) for evaluation. See [parameters](#) for available options and the following sections for how to choose these objectives based of the amount of effective pairs.

- NDCG

Normalized Discounted Cumulative Gain `NDCG` can be used with both binary relevance and multi-level relevance. If you are not sure about your data, this metric can be used as the default. The name for the objective is `rank:ndcg`.

- MAP

Mean average precision `MAP` is a binary measure. It can be used when the relevance label is 0 or 1. The name for the objective is `rank:map`.

- Pairwise

The *LambdaMART* algorithm scales the logistic loss with learning to rank metrics like `NDCG` in the hope of including ranking information into the loss function. The `rank:pairwise` loss is the original version of the pairwise loss, also known as the *RankNet loss* [7] or the *pairwise logistic loss*. Unlike the `rank:map` and the `rank:ndcg`, no scaling is applied ($|\Delta Z_{ij}| = 1$).

Whether scaling with a LTR metric is actually more effective is still up for debate; [8] provides a theoretical foundation for general lambda loss functions and some insights into the framework.

Constructing Pairs

There are two implemented strategies for constructing document pairs for λ -gradient calculation. The first one is the `mean` method, another one is the `topk` method. The preferred strategy can be specified by the `lambdarank_pair_method` parameter.

For the `mean` strategy, XGBoost samples `lambdarank_num_pair_per_sample` pairs for each document in a query list. For example, given a list of 3 documents and `lambdarank_num_pair_per_sample` is set to 2, XGBoost will randomly sample 6 pairs, assuming the labels for these documents are different. On the other hand, if the pair method is set to `topk`, XGBoost constructs about $k \times |query|$ number of pairs with $|query|$ pairs for each sample at the top $k = \text{lambdarank_num_pair}$ position. The number of pairs counted here is an approximation since we skip pairs that have the same label.

Obtaining Good Result

Learning to rank is a sophisticated task and an active research area. It's not trivial to train a model that generalizes well. There are multiple loss functions available in XGBoost along with a set of hyperparameters. This section contains some hints for how to choose hyperparameters as a starting point. One can further optimize the model by tuning these hyperparameters.

The first question would be how to choose an objective that matches the task at hand. If your input data has multi-level relevance degrees, then either `rank:ndcg` or `rank:pairwise` should be used. However, when the input has binary labels, we have multiple options based on the target metric. [6] provides some guidelines on this topic and users are encouraged to see the analysis done in their work. The choice should be based on the number of *effective pairs*, which refers to the number of pairs that can generate non-zero gradient and contribute to training. *LambdaMART* with `MRR` has

the least amount of effective pairs as the λ -gradient is only non-zero when the pair contains a non-relevant document ranked higher than the top relevant document. As a result, it's not implemented in XGBoost. Since `ndcg` is a multi-level metric, it usually generate more effective pairs than `map`.

However, when there are sufficiently many effective pairs, it's shown in [6] that matching the target metric with the objective is of significance. When the target metric is `map` and you are using a large dataset that can provide a sufficient amount of effective pairs, `rank:map` can in theory yield higher `map` value than `rank:ndcg`.

The consideration of effective pairs also applies to the choice of pair method (`lambdarank_pair_method`) and the number of pairs for each sample (`lambdarank_num_pair_per_sample`). For example, the mean-`ndcg` considers more pairs than `ndcg@10`, so the former generates more effective pairs and provides more granularity than the latter. Also, using the `mean` strategy can help the model generalize with random sampling. However, one might want to focus the training on the top k documents instead of using all pairs, to better fit their real-world application.

When using the mean strategy for generating pairs, where the target metric (like `ndcg`) is computed over the whole query list, users can specify how many pairs should be generated per each document, by setting the `lambdarank_num_pair_per_sample`. XGBoost will randomly sample `lambdarank_num_pair_per_sample` pairs for each element in the query group ($|pairs| = |query| \times num_pairsample$). Often, setting it to 1 can produce reasonable results. In cases where performance is inadequate due to insufficient number of effective pairs being generated, set `lambdarank_num_pair_per_sample` to a higher value. As more document pairs are generated, more effective pairs will be generated as well.

On the other hand, if you are prioritizing the top k documents, the `lambdarank_num_pair_per_sample` should be set slightly higher than k (with a few more documents) to obtain a good training result. Lastly, XGBoost employs additional regularization for learning to rank objectives, which can be disabled by setting the `lambdarank_normalization` to `False`.

Summary If you have large amount of training data:

- Use the target-matching objective.
- Choose the `topk` strategy for generating document pairs (if it's appropriate for your application).

On the other hand, if you have comparatively small amount of training data:

- Select `ndcg` or the RankNet loss (`rank:pairwise`).
- Choose the `mean` strategy for generating document pairs, to obtain more effective pairs.

For any method chosen, you can modify `lambdarank_num_pair_per_sample` to control the amount of pairs generated.

Distributed Training

XGBoost implements distributed learning-to-rank with integration of multiple frameworks including [Dask](#), [Spark](#), and [PySpark](#). The interface is similar to the single-node counterpart. Please refer to document of the respective XGBoost interface for details.

Warning

Position-debiasing is not yet supported for existing distributed interfaces.

XGBoost works with collective operations, which means data is scattered to multiple workers. We can divide the data partitions by query group and ensure no query group is split among workers. However, this requires a costly sort and groupby operation and might only be necessary for selected use cases. Splitting and scattering a query group to multiple workers is theoretically sound but can affect the model's accuracy. If there are only a small number of groups sitting at the boundaries of workers, the small discrepancy is not an issue, as the amount of training data is usually large when distributed training is used.

For a longer explanation, assuming the pairwise ranking method is used, we calculate the gradient based on relevance degree by constructing pairs within a query group. If a single query group is split among workers and we use worker-local data for gradient calculation, then we are simply sampling pairs from a smaller group for each worker to calculate the gradient and the evaluation metric. The comparison between each pair doesn't change because a group is split into sub-groups, what changes is the number of total and effective pairs and normalizers like *IDCG*. One can generate more pairs from a large group than it's from two smaller subgroups. As a result, the obtained gradient is still valid from a theoretical standpoint but might not be optimal. As long as each data partitions within a worker are correctly sorted by query IDs, XGBoost can aggregate sample gradients accordingly. And both the (Py)Spark interface and the Dask interface can sort the data according to query ID, please see respected tutorials for more information.

However, it's possible that a distributed framework shuffles the data during map reduce and splits every query group into multiple workers. In that case, the performance would be disastrous. As a result, it depends on the data and the framework for whether a sorted groupby is needed.

Reproducible Result

Like any other tasks, XGBoost should generate reproducible results given the same hardware and software environments (and data partitions, if distributed interface is used). Even when the underlying environment has changed, the result should still be consistent. However, when the `lambdarank_pair_method` is set to `mean`, XGBoost uses random sampling, and results may differ depending on the platform used. The random number generator used on Windows (Microsoft Visual C++) is different from the ones used on other platforms like Linux (GCC, Clang)^[1], so the output varies significantly between these platforms.

[1] `minstd_rand` implementation is different on MSVC. The implementations from GCC and Thrust produce the same output.

References

[1] Tie-Yan Liu. 2009. "Learning to Rank for Information Retrieval". Found. Trends Inf. Retr. 3, 3 (March 2009), 225–331.

- [2] Christopher J. C. Burges, Robert Ragno, and Quoc Viet Le. 2006. "[Learning to rank with nonsmooth cost functions](#)". In Proceedings of the 19th International Conference on Neural Information Processing Systems (NIPS'06). MIT Press, Cambridge, MA, USA, 193–200.
- [3] Wu, Q., Burges, C.J.C., Svore, K.M. et al. "[Adapting boosting for information retrieval measures](#)". Inf Retrieval 13, 254–270 (2010).
- [4] Ziniu Hu, Yang Wang, Qu Peng, Hang Li. "[Unbiased LambdaMART: An Unbiased Pairwise Learning-to-Rank Algorithm](#)". Proceedings of the 2019 World Wide Web Conference.
- [5] Burges, Chris J.C. "[From RankNet to LambdaRank to LambdaMART: An Overview](#)". MSR-TR-2010-82
- [6] Pinar Donmez, Krysta M. Svore, and Christopher J.C. Burges. 2009. "[On the local optimality of LambdaRank](#)". In Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval (SIGIR '09). Association for Computing Machinery, New York, NY, USA, 460–467.
- [7] Chris Burges, Tal Shaked, Erin Renshaw, Ari Lazier, Matt Deeds, Nicole Hamilton, and Greg Hullender. 2005. "[Learning to rank using gradient descent](#)". In Proceedings of the 22nd international conference on Machine learning (ICML '05). Association for Computing Machinery, New York, NY, USA, 89–96.
- [8] Xuanhui Wang and Cheng Li and Nadav Golbandi and Mike Bendersky and Marc Najork. 2018. "[The LambdaLoss Framework for Ranking Metric Optimization](#)". Proceedings of The 27th ACM International Conference on Information and Knowledge Management (CIKM '18).